GPS Tracking System

Yiheng Shan a1898063

1. Purpose and scope

This report describes the results of functional, FRP, distance, boundary and GUI testing of the Java GPS Tracking System. It explains how to compile and run the program and provides test results in the form of videos and/or screenshots of the application and test programs. The program uses simulated GPS events from the file GpsService and gps.dat.

2. Environment and execution

Item	Environment / setup
Language / compiler	Java; assessment specifies JDK 11 (local installed version not independently verified)
FRP dependency	Sodium Java JAR: lib/sodium-1.2.0.jar
GUI	Java Swing
Runtime input	gps.dat located in the working directory
Working directory	Directory containing GpsGUI.java and related Java files

Windows PowerShell - compile:

javac -cp ".;lib/sodium-1.2.0.jar" GpsGUI.java GpsGUI_Test.java GpsFRP_Test.java GpsBoundary_Test.java GpsEvent.java GpsService.java

Run GUI: java -cp ".;lib/sodium-1.2.0.jar" GpsGUI

Run tests (one at a time):

java -cp ".;lib/sodium-1.2.0.jar" GpsGUI_Test
java -cp ".;lib/sodium-1.2.0.jar" GpsFRP_Test
java -cp ".;lib/sodium-1.2.0.jar" GpsBoundary_Test

For the separate timer demonstration: javac GpsTimer_Test.java; then java GpsTimer_Test.

3. Recorded test summary

Test group	Passed	Failed	Evidence
GpsGUI_Test - distance and filter logic	12	0	Figure 2
GpsFRP_Test - simulated stream operations	10	0	Figure 3
GpsBoundary_Test - sliding-window edge cases	6	0	Figure 4
Total automated cases	28	0	Terminal screenshots
GpsTimer_Test - component demonstration	Not counted	Not counted	Figures 8-9; manual / console observation

All 28 named automated test cases show PASS in the supplied terminal screenshots. This total does not include the separate timer demonstration or GUI screenshots.

4. Functional tests and expected behaviour

ID	Test / procedure	Expected outcome	Evidence / status
F01	Start GpsGUI with gps.dat in the working directory	Ten tracker coordinate rows display and update	GUI visible - Fig. 1
F02	Read simplified tracker coordinates	Name, latitude, longitude; altitude excluded	Visible - Fig. 1
F03	Default filter: latitude -90, longitude -180	Normal events can appear in filtered latest-event area	Observed - Fig. 1
F04	Apply latitude 39.9 and longitude 116.3	Latest accepted event satisfies both minimum thresholds	Observed - Fig. 5
F05	Enter text abc as latitude; press Set Filter	Error dialog, active filter remains unchanged	Observed - Fig. 6
F06	Apply extreme limits latitude 90, longitude 180	No observed matching events; distance histories reset	Observed - Fig. 7
F07	Send event, then stop sending for 3 seconds	Latest-event label clears after timer expires	Separate component test - Figs. 8-9
F08	Wait for events and observe five-minute totals	Each tracker has own meter distance display	Display observed; accuracy covered by automated cases
F09	Reset filter during accumulated movement	History restarts with new setting	Automated state tests; manual screenshot not supplied

5. Automated test coverage

5.1 GPS calculation and state logic

GpsGUI_Test passed 12 cases: feet-to-meters conversion, zero distance, vertical distance, latitude distance, longitude distance, 3D distance, GPS filter conditions, cumulative segment distance, five-minute window expiry, filter reset, old-version rejection and upward rounding. These tests exercise private calculation/state classes using Java reflection.

5.2 FRP stream operations

GpsFRP_Test passed 10 cases: zero, one and two events, simplified map, merge of two streams, default filter, latitude filter, longitude filter, combined filter and dynamic filter changes. These use separately constructed simulated Sodium streams, rather than directly invoking the full GUI.

5.3 Time boundaries and tracker independence

GpsBoundary_Test passed 6 cases: exactly five minutes, one millisecond beyond the window, independent tracker distances, reset of all trackers, ignoring previous filter versions and a single GPS position. The chosen policy retains positions at the exact five-minute boundary and removes earlier positions.

6. Implementation notes

The simplified coordinate display is produced using Sodium map(); filtered data uses snapshot() and filter(); merged streams use orElse(); altitude conversion uses map(); distance state is updated with collect(). The GUI uses Swing and updates labels on the Swing event dispatch thread.

Distance is calculated as the local three-dimensional displacement using latitude, longitude and altitude in meters. Only pairs of retained known positions are summed. The GUI applies Math.ceil to the final meter total. The distance display is a rolling five-minute view, not an all-time sum.

7. Screenshot evidence

Figure 1. Application overview

The running GUI shows ten tracker rows, five-minute distances, minimum-coordinate filter controls, a recent filtered event and converted altitude. Under the default filter (-90, -180), the screenshot shows an accepted Tracker6 event and its converted altitude.

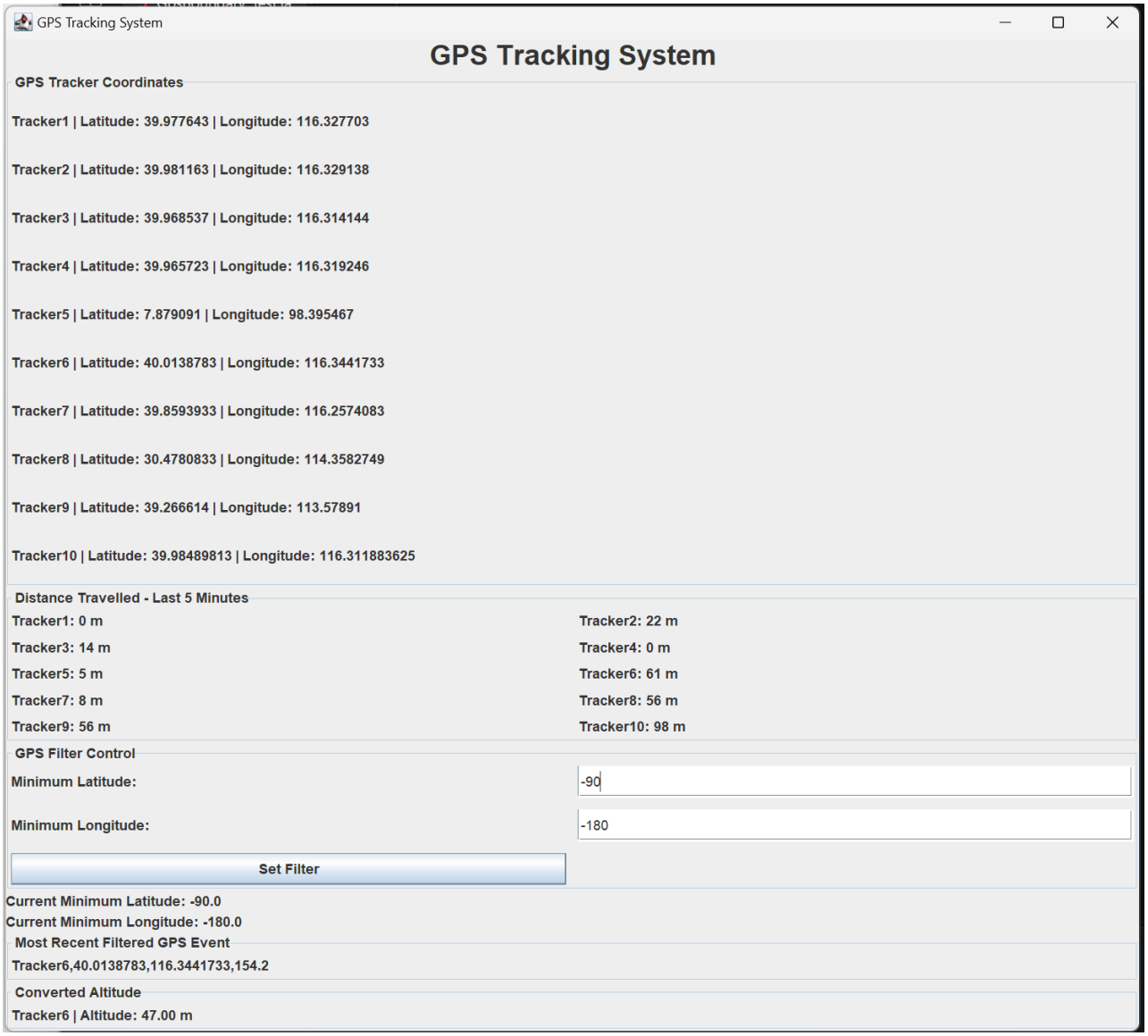


Figure 1 - Main GUI under default filtering.

Figure 2. Distance and filtering unit tests

```
PS C:\Users\yihengshan\Desktop\GpsFiles> java -cp ".;lib/sodium-1.2.0.jar" GpsGUI_Test
GPS Assignment 3 - Tests
------------------------
PASS: Altitude feet to meters
PASS: Zero distance
PASS: Vertical distance
PASS: Latitude distance
PASS: Longitude distance
PASS: Three-dimensional distance
PASS: GPS filter conditions
PASS: Cumulative segment distance
PASS: Five-minute window expiry
PASS: Filter reset
PASS: Old filter version ignored
PASS: Distance rounding
------------------------
Passed: 12
Failed: 0
Total: 12
```

Figure 2 - GpsGUI_Test: 12 passed, 0 failed.

Figure 3. Simulated Sodium FRP tests

```
PS C:\Users\yihengshan\Desktop\GpsFiles> java -cp ".;lib/sodium-1.2.0.jar" GpsFRP_Test
GPS FRP Integration Tests
-------------------------
PASS: Zero GPS events
PASS: One GPS event
PASS: Two GPS events
PASS: Simplified GPS map
PASS: Merge two GPS streams
PASS: Default GPS filter
PASS: Latitude filtering
PASS: Longitude filtering
PASS: Combined GPS filtering
PASS: Dynamic filter changes
------------------------
Passed: 10
Failed: 0
Total: 10
```

Figure 3 - GpsFRP_Test: 10 passed, 0 failed.

Figure 4. Time-window boundary tests

```
PS C:\Users\yihengshan\Desktop\GpsFiles> java -cp ".;lib/sodium-1.2.0.jar" GpsBoundary_Test
GPS Boundary Tests
------------------
PASS: Exactly five minutes
PASS: One millisecond past boundary
PASS: Independent tracker distances
PASS: Reset clears all trackers
PASS: Ignore previous filter version
PASS: Single GPS position
------------------
Passed: 6
Failed: 0
Total: 6
```

Figure 4 - GpsBoundary_Test: 6 passed, 0 failed.

Figure 5. Filtering by latitude and longitude

The filter values are minimum latitude 39.9 and minimum longitude 116.3. The displayed filtered event has latitude 40.0095066 and longitude 116.3460516, satisfying both conditions. Individual tracker rows still show unfiltered simplified coordinates by design.

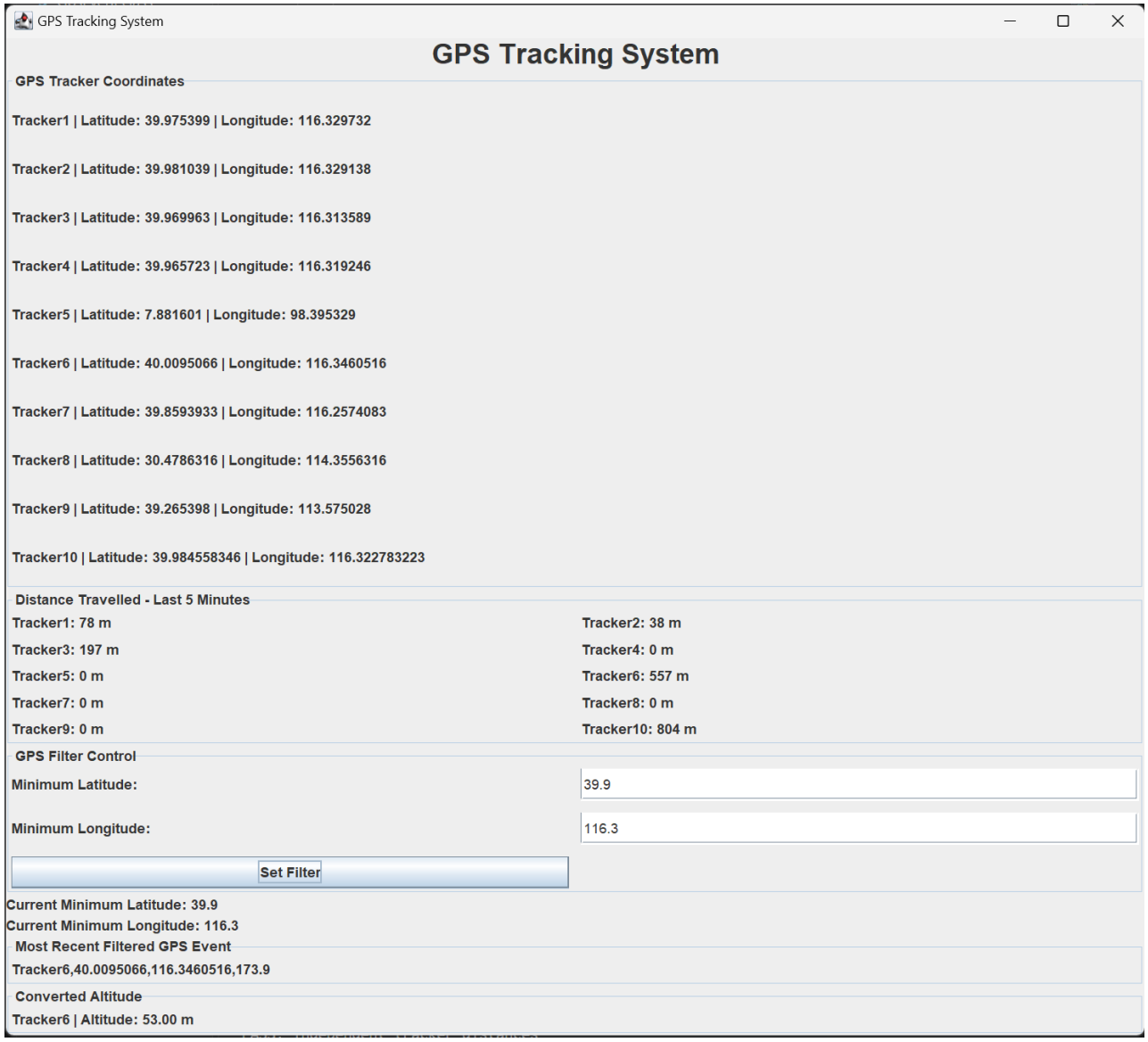


Figure 5 - GUI after applying both coordinate restrictions.

Figure 6. Invalid input handling

The latitude input contains abc, and the program displays the Invalid Input dialog with valid coordinate ranges. The current active limits remain 39.9 and 116.3 in the screenshot.

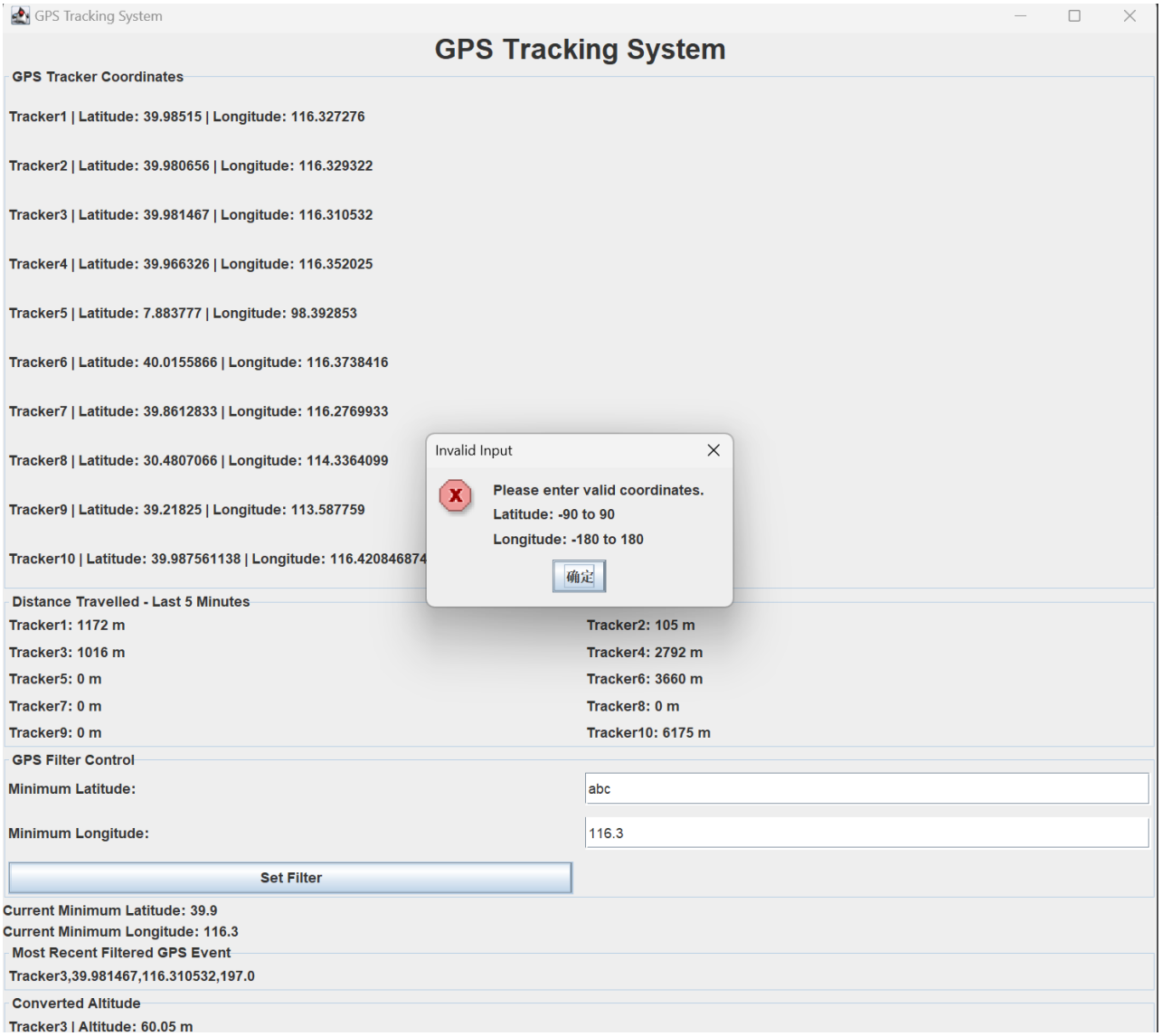


Figure 6 - Non-numeric latitude is rejected; previous settings remain displayed.

Figure 7. Strict filter setting

With latitude 90 and longitude 180, the latest filtered-event display is empty, the ten distances are zero after filter reset and altitude is waiting for data. This demonstrates filter state, not the independent three-second timeout.

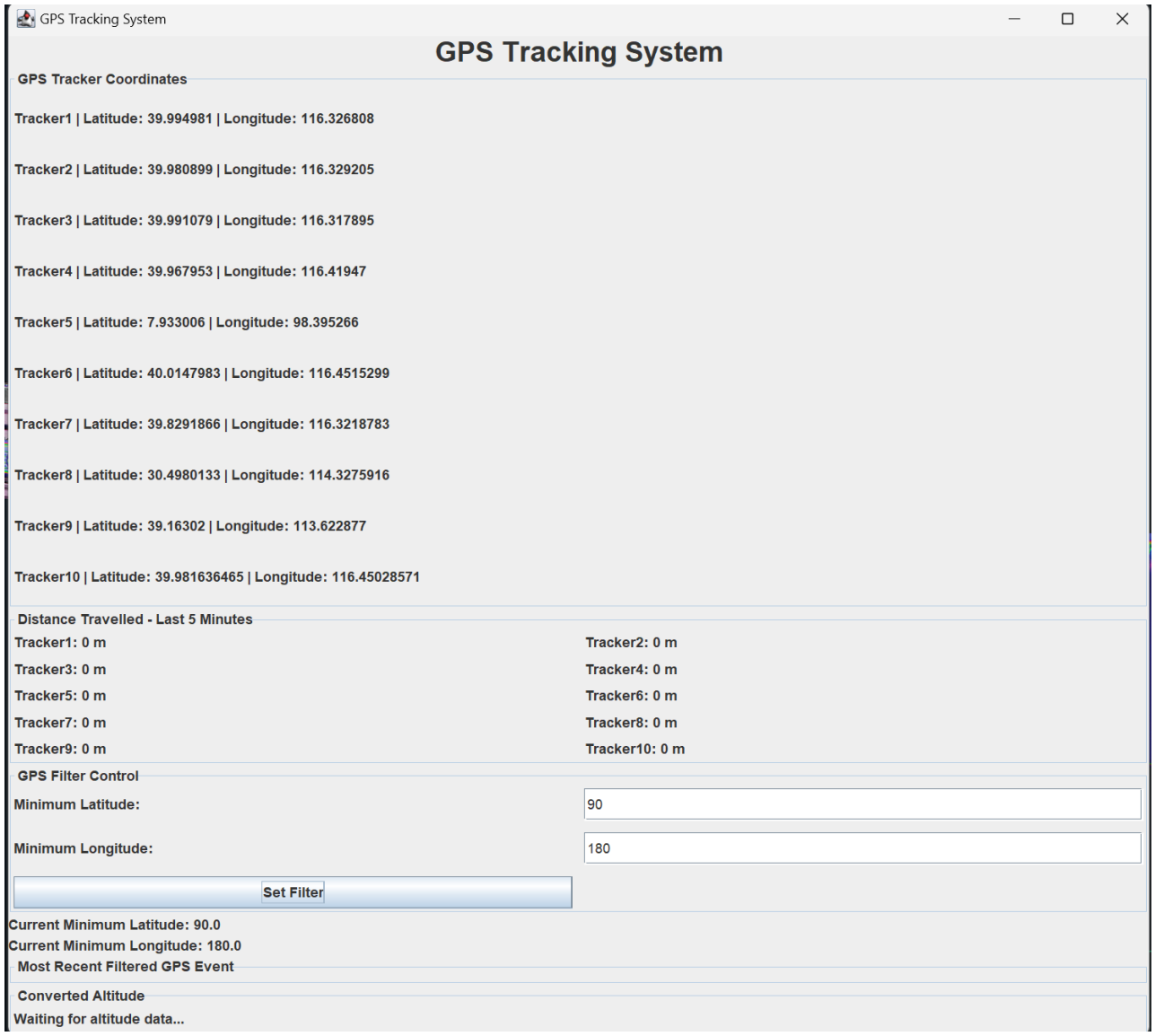


Figure 7 - Strict filter (90, 180), no currently accepted event.

Figure 8. Separate three-second timeout component

The standalone GpsTimer_Test window uses a one-shot Swing Timer with a 3000 ms delay. The screenshot shows the event label empty after the timer event. As this is a separate test application, it does not alone establish end-to-end timeout operation in the main GpsGUI.

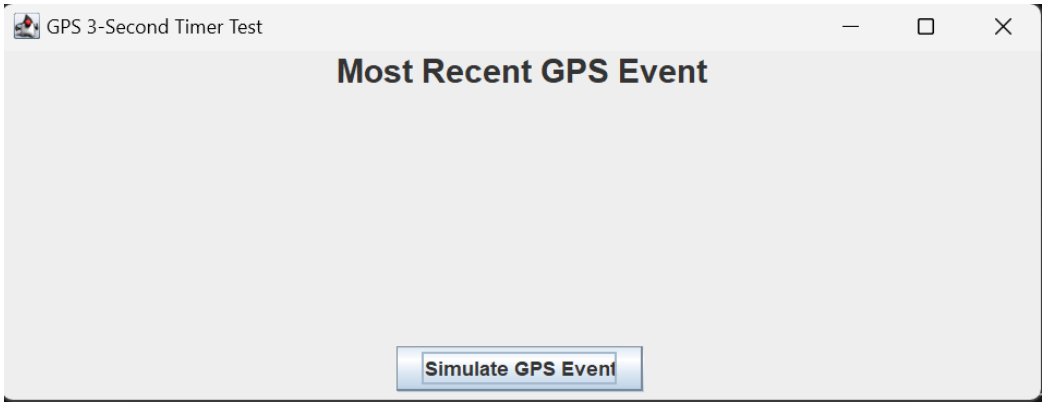


Figure 8 - GpsTimer_Test window after the event label has cleared.

Figure 9. Timeout demonstration console output

The console records repeated simulated event submissions followed by a timer callback message. This is observational evidence of the timer callback; the message is printed by the test code and is not a measured timing assertion.

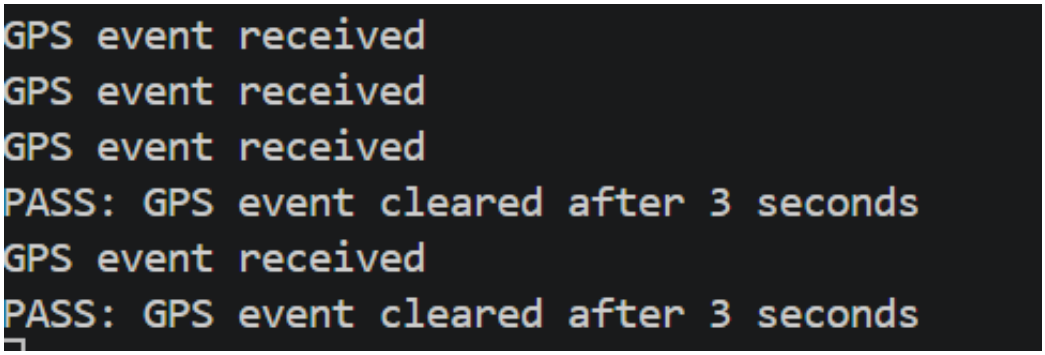


Figure 9 - Simulated GPS event and clearing messages in the terminal.

VEDIO:

https://youtu.be/k1-N8dSTfNo